

Azure Service Bus – Dead-Letter Queue

1. What is a Dead-Letter Queue?

A **Dead-Letter Queue (DLQ)** is a special sub-queue associated with an Azure Service Bus queue.

It is used to store messages that should not continue through normal processing.

For this example, we will learn how to:

1. Move a message to the Dead-Letter Queue from C#.
2. Create a receiver for the Dead-Letter Queue.
3. Read the dead-lettered message.

2. Moving a Message to Dead-Letter Queue

Suppose the queue contains:

```
Error Message let's move it to dead letter
```

Initially:

```
Queue Messages      = 1  
Dead-Letter Messages = 0
```

The application receives the message:

```
ServiceBusReceivedMessage message =  
    await receiver.ReceiveMessageAsync();
```

Instead of completing the message, we move it to the Dead-Letter Queue.

```
await receiver.DeadLetterMessageAsync(message);
```

Important Code

```
await receiver.DeadLetterMessageAsync(message);
```

This moves the message from the normal queue to its Dead-Letter Queue.

3. What Happens After Dead-Lettering?

Before:

```
Main Queue
  ↓
1 Message

Dead-Letter Queue
  ↓
0 Messages
```

After:

```
Main Queue
  ↓
0 Messages

Dead-Letter Queue
  ↓
1 Message
```

The message is not deleted. It has been moved to the Dead-Letter Queue.

4. Verify in Azure Portal

In Azure Service Bus Explorer, refresh the queue after executing:

```
await receiver.DeadLetterMessageAsync(message);
```

You should see:

```
Queue Messages      = 0
Dead-Letter Messages = 1
```

Open the Dead-Letter section and use **Pick from start**.

The original message should be visible there.

Example:

```
Error Message let's move it to dead letter
```

5. Reading a Message from the Dead-Letter Queue

The Dead-Letter Queue is associated with the original queue.

To read from it, create a receiver with the `DeadLetter` sub-queue option.

```
ServiceBusReceiver deadLetterReceiver =
    client.CreateReceiver(
        queueName,
        new ServiceBusReceiverOptions
        {
            SubQueue = SubQueue.DeadLetter
        });
```

Important Concept

We are still providing the **original queue name**.

The additional option tells the client:

```
Create the receiver for the Dead-Letter sub-queue of this queue.
```

6. Receive the Dead-Letter Message

Once the receiver is created:

```
ServiceBusReceivedMessage deadLetterMessage =  
    await deadLetterReceiver.ReceiveMessageAsync();
```

Now the variable:

```
deadLetterMessage
```

contains the message retrieved from the Dead-Letter Queue.

We can inspect its body:

```
Console.WriteLine(  
    deadLetterMessage.Body.ToString());
```

Output:

```
Error Message let's move it to dead letter
```

This confirms that the message was successfully retrieved from the DLQ.

7. Complete Flow

```
Main Queue  
  ↓  
Receive Message  
  ↓  
Problematic Message  
  ↓  
DeadLetterMessageAsync()  
  ↓  
Dead-Letter Queue  
  ↓  
Create DLQ Receiver  
  ↓  
ReceiveMessageAsync()  
  ↓  
Read Dead-Letter Message
```

8. Complete Example

```
using Azure.Messaging.ServiceBus;

class Program
{
    static async Task Main(string[] args)
    {
        string connectionString = "YOUR_CONNECTION_STRING";
        string queueName = "YOUR_QUEUE_NAME";

        ServiceBusClient client =
            new ServiceBusClient(connectionString);

        // Receiver for the normal queue
        ServiceBusReceiver receiver =
            client.CreateReceiver(queueName);

        // Receive message from normal queue
        ServiceBusReceivedMessage message =
            await receiver.ReceiveMessageAsync();

        Console.WriteLine(
            message.Body.ToString());

        // Move message to Dead-Letter Queue
        await receiver.DeadLetterMessageAsync(message);

        // Create receiver for Dead-Letter Queue
        ServiceBusReceiver deadLetterReceiver =
            client.CreateReceiver(
                queueName,
                new ServiceBusReceiverOptions
                {
                    SubQueue = SubQueue.DeadLetter
                });

        // Receive message from Dead-Letter Queue
        ServiceBusReceivedMessage deadLetterMessage =
            await deadLetterReceiver.ReceiveMessageAsync();

        Console.WriteLine(
            deadLetterMessage.Body.ToString());
    }
}
```

9. Important Difference: Normal Queue vs Dead-Letter Queue

Normal Queue Receiver

```
ServiceBusReceiver receiver =  
    client.CreateReceiver(queueName);
```

Reads messages from the normal queue.

Dead-Letter Queue Receiver

```
ServiceBusReceiver deadLetterReceiver =  
    client.CreateReceiver(  
        queueName,  
        new ServiceBusReceiverOptions  
        {  
            SubQueue = SubQueue.DeadLetter  
        }  
    );
```

Reads messages from the Dead-Letter Queue.

The important difference is:

```
SubQueue = SubQueue.DeadLetter
```

10. Why Do We Read Messages from the DLQ?

Messages in the Dead-Letter Queue can be inspected to understand why they were not successfully processed.

Typical activities include:

- Debugging
- Investigating failures
- Checking invalid messages
- Identifying processing problems
- Deciding whether a message should be reprocessed

The DLQ therefore acts as a place where problematic messages can be investigated instead of being lost.

11. Real-Time Example

Consider an **Order Processing Service**.

A message contains an order:

```
{
  "OrderId": 1001,
  "Amount": 5000
}
```

Suppose the application cannot process this message because of a permanent problem.

The application can move it to the DLQ:

```
await receiver.DeadLetterMessageAsync(message);
```

Later, a support or background process can read the message from the DLQ:

```
ServiceBusReceiver deadLetterReceiver =
    client.CreateReceiver(
        queueName,
        new ServiceBusReceiverOptions
        {
            SubQueue = SubQueue.DeadLetter
        }
    );
```

Then:

```
var message =
    await deadLetterReceiver.ReceiveMessageAsync();
```

The team can inspect the message and determine what went wrong.

12. Important Interview Questions

Q1. What is a Dead-Letter Queue?

Answer:

A Dead-Letter Queue is a special sub-queue associated with an Azure Service Bus queue where messages that cannot or should not be processed normally can be stored for further investigation or handling.

Q2. How do you move a message to the Dead-Letter Queue?

```
await receiver.DeadLetterMessageAsync(message);
```

Q3. How do you create a receiver for the Dead-Letter Queue?

```
var deadLetterReceiver =  
    client.CreateReceiver(  
        queueName,  
        new ServiceBusReceiverOptions  
        {  
            SubQueue = SubQueue.DeadLetter  
        }  
    );
```

Q4. How do you receive a message from the DLQ?

```
var message =  
    await deadLetterReceiver.ReceiveMessageAsync();
```

Q5. Is the Dead-Letter Queue a completely separate Service Bus queue?

Answer:

No. For this scenario, think of it as a **special sub-queue associated with the original queue**. We use `SubQueue.DeadLetter` to create a receiver for it.

13. Quick Revision

```
MAIN QUEUE
  ↓
Receive Message
  ↓
Problem / Cannot Process
  ↓
DeadLetterMessageAsync()
  ↓
DEAD-LETTER QUEUE
  ↓
Create Receiver with
SubQueue.DeadLetter
  ↓
ReceiveMessageAsync()
  ↓
Read / Investigate Message
```

Most Important Code

Move to DLQ:

```
await receiver.DeadLetterMessageAsync(message);
```

Create DLQ Receiver:

```
var deadLetterReceiver =
    client.CreateReceiver(
        queueName,
        new ServiceBusReceiverOptions
        {
            SubQueue = SubQueue.DeadLetter
        });
```

Read from DLQ:

```
var message =
    await deadLetterReceiver.ReceiveMessageAsync();
```

Easy Memory Trick

Dead-Letter = Move problematic message aside → Read it later from the DLQ for investigation.